

Full-Stack Developer — Take-Home Assignment

Real-Time Collaborative Workspace with Role-Based Access Control

Level: **Advanced** · Estimated Effort: **20–30 hours** · Submission Window: **10 calendar days** from receipt · Role: Full-Stack Developer (Senior / Advanced)

Tags

■ TypeScript ■ React / Next.js ■ Node.js ■ Express or NestJS ■ PostgreSQL ■ Prisma ■ Redis ■ WebSockets ■ Docker ■ GitHub Actions (CI) ■ JWT + Refresh Tokens ■ Vercel ■ Render

Overview

This assignment is deliberately closer to a real sprint ticket than a tutorial exercise. You will design and build a **multi-tenant, real-time collaborative workspace** — think a scaled-down Linear or Asana — where multiple organizations, each with their own members and permission levels, manage boards and tasks together and see each other's changes appear live, without refreshing the page.

The functional surface area is intentionally larger than a CRUD app: you'll need to reason about concurrency (two people editing the same task at once), authorization (who is allowed to do what, and where that's enforced), performance under repeated reads, and what happens when a dependent service (the database, the cache, the socket connection) is briefly unavailable. We're evaluating engineering judgment as much as feature completion.

Aim of the Project

- Evaluate your ability to design a multi-tenant data model and enforce authorization boundaries consistently across the stack — not just hide UI elements.
- Assess your handling of real-time state: concurrent edits, race conditions, and keeping multiple connected clients in sync without polling.
- See how you make architectural trade-offs under partial ambiguity — caching strategy, queue usage, schema normalization — and whether you can explain the reasoning.
- Confirm you can build for production conditions: automated tests, CI, containerized local dev, and a live deployment that survives a cold start.

Functional Requirements

- **Multi-tenant workspaces** — A user can create a workspace (organization) or be invited into one by email. All data (boards, tasks, members) is scoped to a workspace; there must be no way for one workspace's data to leak into another via the API.
- **Role-based access control** — Roles of Owner, Admin, Member, and Viewer, each with distinct permissions (e.g. only Owner/Admin can invite or remove members; Viewer is read-only). Enforce this server-side on every mutating endpoint, not just in the UI.
- **Boards, lists & tasks** — Workspaces contain boards; boards contain ordered lists (columns); lists contain ordered tasks. Task order and list order must persist correctly under concurrent drag-and-drop from different users.

- **Real-time sync** — When one user creates, edits, moves, or deletes a task, every other connected client viewing that board reflects the change within roughly a second, via WebSockets (Socket.IO, native ws, or equivalent) — not polling.
- **Authentication** — Email/password signup and login with hashed passwords (bcrypt/argon2), short-lived access tokens, and a refresh-token rotation flow. Explain your token storage and revocation strategy in the README.
- **Activity log** — Every meaningful mutation (task created/moved/deleted, member added/removed, role changed) is recorded and queryable per workspace, with actor, action, and timestamp.
- **Search & filtering** — Full-text search over task titles/descriptions plus filtering by assignee, label, and status, returned paginated (not the entire dataset in one response).
- **Caching** — At least one genuinely expensive read path (e.g. a workspace dashboard/summary endpoint) is backed by Redis with a sane invalidation strategy, not cached forever or never invalidated.
- **Background processing** — At least one operation (e.g. a daily digest email, a notification, an export job) runs asynchronously via a job queue (BullMQ, or equivalent) rather than blocking the request that triggered it.
- **Validation & error handling** — Consistent request validation, meaningful HTTP status codes, and no unhandled promise rejections or stack traces leaking to the client.

Stretch goals (optional): presence indicators (who's currently viewing a board), optimistic UI updates with conflict resolution, rate limiting per user/IP, or exporting a board to CSV/PDF via the background queue. Only attempt these once every requirement above is solid.

Suggested Steps

- **1. Design before coding.** Write a short design note (half a page is fine): entity-relationship diagram covering workspaces/users/roles/boards/tasks, and how authorization is enforced at the API layer. This is the single highest-leverage step — rushing it costs you later.
- **2. Stand up the environment.** Docker Compose for Postgres, Redis, and the app itself, so `docker compose up` gets a contributor running locally with no manual setup.
- **3. Build auth and tenancy first.** Signup/login, refresh-token flow, workspace creation/invites, and role enforcement — everything else depends on this being correct.
- **4. Build the core REST/GraphQL API.** Boards, lists, tasks, ordering, search, pagination. Write integration tests against a real (test) database as you go, not after.
- **5. Add the real-time layer.** Wire WebSocket events for the mutations above; verify with two browser windows logged in as different users in the same workspace.
- **6. Add caching and the background queue.** Pick one expensive read for Redis and one side-effect for the job queue; be ready to justify why those two.
- **7. Build the frontend.** React/Next.js consuming the real API and socket events — board view, drag-and-drop, member/role management, activity log, search.
- **8. Set up CI.** A GitHub Actions workflow that installs dependencies, runs lint, and runs your test suite on every push.
- **9. Deploy it.** Frontend to **Vercel**, backend (API + WebSocket server) to **Render**, with managed or hosted Postgres and Redis. Confirm the deployed frontend talks to the deployed backend in real time, not localhost.
- **10. Write the README.** Architecture diagram or description, setup instructions, environment variables, the trade-offs you made under time pressure, and what you'd do next with more time.

Technical Requirements

- Language: TypeScript strongly preferred on both frontend and backend.
- Frontend: React or Next.js.
- Backend: Node.js with Express or NestJS, exposing REST or GraphQL, plus a WebSocket server.
- Database: PostgreSQL with a schema-first ORM (Prisma or TypeORM) and versioned migrations checked into the repo.
- Cache / queue: Redis, used for both response caching and backing the background job queue.
- Containerization: a working **Dockerfile** per service and a **docker-compose.yml** that brings up the full stack locally.
- CI: GitHub Actions running lint and the automated test suite on every push.
- Testing: unit tests for business logic (authorization rules, ordering logic) plus integration tests for at least the auth and task-mutation endpoints.
- Deployment: frontend live on **Vercel**, backend (API + WebSockets) live on **Render**, database and Redis hosted (Render managed Postgres/Redis, Neon + Upstash, or equivalent).
- Source control: a public (or shared-access) GitHub repository with an incremental commit history that reflects how the system was actually built.

Submission Guidelines

- GitHub repository link, including the CI workflow file and passing build badge/status.
- Live Vercel URL for the frontend and live Render URL for the backend API/WebSocket server.
- A short design note or README section covering the data model, authorization enforcement, caching/queue choices, and known limitations.
- Two working test accounts in different roles (e.g. an Owner and a Member of the same workspace) so we can verify RBAC without asking you for credentials.
- Instructions to reproduce the local Docker Compose environment from a clean checkout.

Evaluation Criteria

Area	What we're looking for	Weight
Architecture & data design	Sound multi-tenant schema, sensible normalization, consistent authorization enforcement.	25%
Real-time correctness	Concurrent edits and ordering behave correctly across multiple connected clients.	20%
Code quality & testing	Clean structure, meaningful automated tests, CI actually passing.	20%
Security	Auth/token handling, authorization checks server-side, no obvious injection/XSS gaps.	15%

Systems thinking	Justified use of caching and background jobs; graceful handling of dependency failures.	10%
Deployment & communication	Both services live and connected; README/design note clearly explains decisions.	10%

Expected Outcome

By the end of this assignment you will have shipped a system that behaves like a small piece of production software: multiple organizations use it in isolation from one another, permissions are enforced consistently rather than assumed, several people can work on the same board at once and see each other's changes live, and the system stays coherent under a restart or a cold cache. It will be reachable at two public URLs — a Vercel frontend and a Render backend — with CI green on the latest commit.

Conclusion

This is an advanced assignment and is scored relative to that bar — we expect trade-offs, and we'd rather see a smaller set of requirements implemented rigorously (correct authorization, real concurrency handling, tests that actually catch regressions) than every box checked shallowly. Where a requirement is ambiguous, make the call a senior engineer would make on a real ticket: document the decision briefly and move forward. How you reason about the parts we didn't fully specify is a core part of what this assignment is measuring.

Questions about the brief? Reply to the email this was sent with and we'll clarify — asking a good question is a positive signal, not a negative one.